

Informe

Sistema de Liquidaciones de Tarjetas de Crédito

Índice

Sistema de Liquidaciones de Tarjetas de Crédito

Índice

1. Introducción y contexto

¿Qué hace este sistema?

¿Qué tipo de aplicación es?

¿Por qué es relevante entender esto?

2. Arquitectura general del sistema

¿Qué hace cada capa?

3. Tecnologías utilizadas

Java 17

Maven

JPA (Jakarta Persistence API)

Hibernate

H2

Lombok

JUnit 5

4. Archivo de configuración Maven - `pom.xml`

¿Qué es un pom.xml?

Estructura del archivo

Identificación del proyecto

Propiedades

Dependencias explicadas una por una

JUnit 5

Hibernate

H2

Lombok

5. Base de datos

5.1 `schema.sql` - Estructura

Tabla COTIZACIONES

Tabla TARJETAS

Tabla CONSUMOS

Tabla LIQUIDACIONES

Diagrama de relaciones entre tablas

5.2	<code>data.sql</code>	- Datos iniciales
	<u>Cotizaciones</u>	
	<u>Tarjetas</u>	
	<u>Consumos</u>	
6.	Configuración JPA - <code>persistence.xml</code>	
	<u>¿Qué es y dónde vive?</u>	
	<u>Análisis completo del archivo</u>	
	<u>Declaración de la unidad de persistencia</u>	
	<u>Registro de entidades</u>	
	<u>Configuración de conexión</u>	
	<u>Configuración de recreación de la BD</u>	
	<u>Configuración de Hibernate</u>	
7.	Capa de Modelo	
	<u>Anotaciones JPA fundamentales</u>	
	<u>Anotaciones Lombok fundamentales</u>	
7.1	<code>Cotizacion.java</code>	
	<u>Puntos clave</u>	
7.2	<code>Tarjeta.java</code>	
	<u>Puntos clave</u>	
7.3	<code>Consumo.java</code>	
	<u>Puntos clave</u>	
7.4	<code>Liquidacion.java</code>	
	<u>Puntos clave</u>	
8.	Capa de Repositorio	
	<u>El EntityManager</u>	
	<u>Las transacciones</u>	
	<u>JPQL vs SQL</u>	
8.1	<code>TarjetaRepository.java</code>	
	<u>CRUD básico</u>	
	<u>Consultas específicas</u>	
8.2	<code>ConsumoRepository.java</code>	
	<u>Consulta específica principal</u>	
8.3	<code>CotizacionRepository.java</code>	
8.4	<code>LiquidacionRepository.java</code>	
	<u>Consulta específica principal</u>	
9.	Punto de entrada - <code>Main.java</code>	
	<u>El ciclo de vida de JPA</u>	
10.	Tests - <code>TarjetasTest.java</code>	
	<u>Estructura general</u>	
	<u>Ciclo de vida de los tests</u>	
	<u>Por qué el orden importa</u>	

Análisis de tests importantes

11. Flujo completo de ejecución

Desde que se ejecuta el programa hasta que termina

12. Relaciones entre componentes

Diagrama de dependencias

Resumen de responsabilidades

1. Introducción y contexto

¿Qué hace este sistema?

Este sistema simula el **backend de una empresa de tarjetas de crédito**. Su función principal es gestionar:

- Las tarjetas de crédito y sus titulares
- Los consumos realizados con cada tarjeta
- Las cotizaciones de monedas extranjeras
- Las liquidaciones mensuales (el resumen de lo que debe pagar cada titular)

¿Qué tipo de aplicación es?

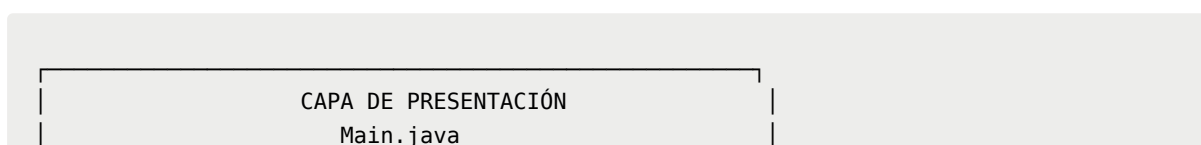
Es una aplicación **Java de consola**, sin interfaz gráfica ni web. No usa Spring Boot. Es un proyecto Maven puro que utiliza JPA con Hibernate para persistir datos en una base de datos H2 en memoria.

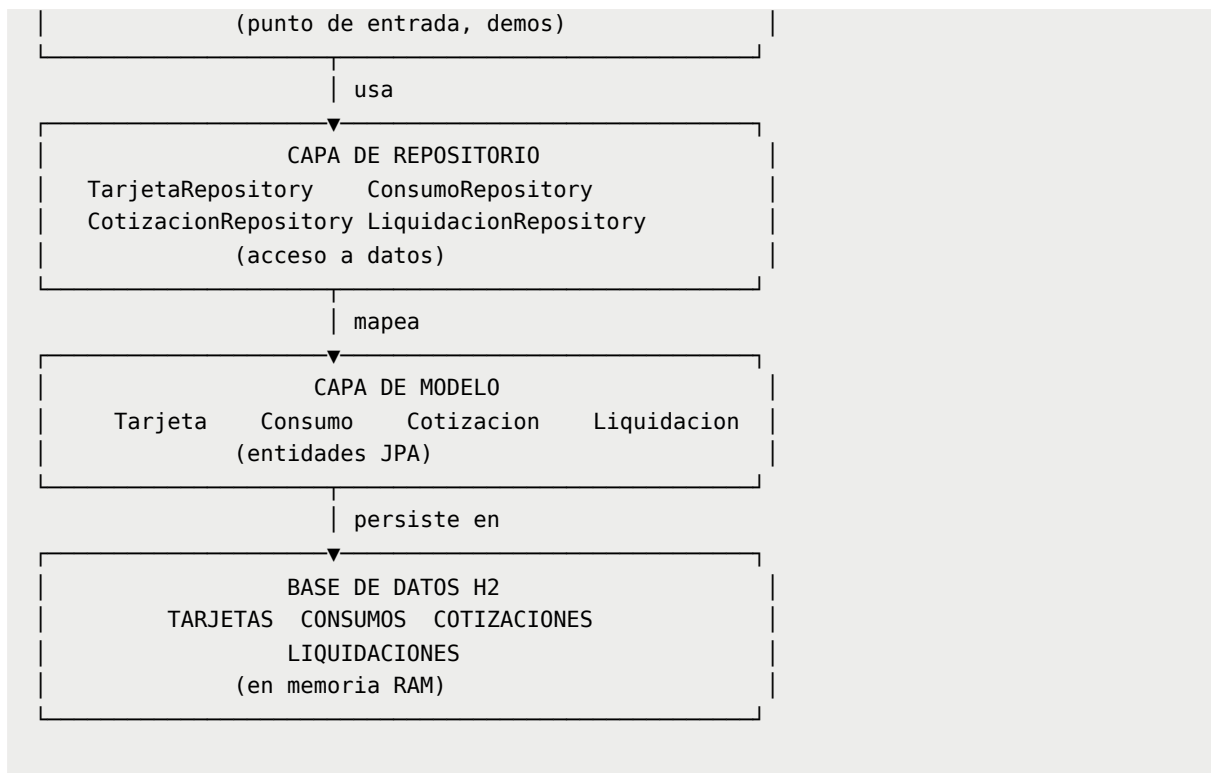
¿Por qué es relevante entender esto?

Este tipo de arquitectura es la **base de cualquier sistema empresarial**. Antes de aprender frameworks como Spring Boot, es fundamental entender cómo funciona JPA directamente, cómo se mapean entidades, cómo se gestionan transacciones y cómo se accede a los datos.

2. Arquitectura general del sistema

El sistema sigue una arquitectura de **dos capas principales**:





¿Qué hace cada capa?

Capa	Responsabilidad
Presentación	Coordina el flujo, muestra resultados
Repositorio	Contiene toda la lógica de acceso a datos
Modelo	Define la estructura de los datos
Base de datos	Almacena físicamente los datos

3. Tecnologías utilizadas

Java 17

La versión del lenguaje. Aporta características modernas como los **text blocks** (las cadenas multilinea con `"""`) que se usan en las consultas JPQL de los repositorios.

```
// Text block de Java 17 - permite escribir SQL/JPQL en múltiples líneas
String jpql = """
    SELECT c FROM Consumo c
```

```
WHERE c.tarjeta.id = :idTarjeta
      AND c.anio = :anio
      """;
```

Maven

Es la herramienta de **gestión del proyecto**. Se encarga de:

- Descargar las dependencias (Hibernate, H2, etc.)
- Compilar el código
- Ejecutar los tests
- Empaquetar la aplicación

JPA (Jakarta Persistence API)

Es una **especificación**, es decir, un conjunto de reglas e interfaces que definen cómo debe funcionar un ORM en Java. No es código ejecutable por sí solo, necesita una implementación.

Hibernate

Es la **implementación de JPA** que usamos. Toma las anotaciones que pusimos en las clases Java y las traduce a SQL para comunicarse con la base de datos.

H2

Es la **base de datos** embebida. Vive dentro de la misma JVM que la aplicación, no requiere instalación externa.

Lombok

Es una biblioteca que **genera código repetitivo automáticamente** en tiempo de compilación. Evita escribir getters, setters, constructores, etc.

```
// Sin Lombok tendrías que escribir esto:
public String getTitular() { return titular; }
public void setTitular(String titular) { this.titular = titular; }
// ... y así para cada campo
```

```
// Con Lombok solo escribís:  
@Getter @Setter  
private String titular;
```

JUnit 5

Es el **framework de testing**. Permite escribir y ejecutar pruebas automatizadas para verificar que el código funciona correctamente.

4. Archivo de configuración Maven - pom.xml

¿Qué es un pom.xml?

POM significa **Project Object Model**. Es el archivo central de Maven que describe el proyecto y todas sus dependencias. Cuando Maven necesita compilar o ejecutar el proyecto, lee este archivo para saber qué bibliotecas descargar y cómo configurar el proceso.

Estructura del archivo

Identificación del proyecto

```
<groupId>ar.edu.utnfc.backend</groupId>  
<artifactId>tarjetas-parcial</artifactId>  
<version>1.0-SNAPSHOT</version>  
<packaging>jar</packaging>
```

Campo	Significado
groupId	Identificador de la organización (como un paquete Java)
artifactId	Nombre del proyecto
version	Versión actual. SNAPSHOT significa "en desarrollo"
packaging	Tipo de archivo de salida (jar = archivo Java ejecutable)

Propiedades

```
<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>
```

Le dice a Maven que compile el código con Java 17 y que use UTF-8 para los archivos de texto.

Dependencias explicadas una por una

JUnit 5

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-api</artifactId>
  <scope>test</scope>
</dependency>
```

El `<scope>test</scope>` significa que esta dependencia **solo existe durante los tests**, no se incluye en el programa final. Provee las anotaciones como `@Test`, `@BeforeAll`, etc.

Hibernate

```
<dependency>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>6.4.4.Final</version>
</dependency>
```

El motor principal del ORM. Incluye automáticamente la API de JPA (Jakarta Persistence), por eso no necesitamos agregarla por separado.

H2

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <version>2.2.224</version>
</dependency>
```

La base de datos embebida. No tiene `<scope>test</scope>` porque la usamos tanto en la aplicación como en los tests.

Lombok

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <version>1.18.32</version>
  <scope>provided</scope>
</dependency>
```

El `<scope>provided</scope>` significa que Lombok solo se necesita **durante la compilación**. Una vez compilado el código, los getters y setters ya están generados y Lombok no hace falta más.

5. Base de datos

5.1 `schema.sql` - Estructura

Este archivo define la **estructura** de la base de datos. Se ejecuta cada vez que arranca la aplicación, después de que Hibernate borra todas las tablas existentes.

Tabla COTIZACIONES

```
CREATE TABLE COTIZACIONES (
  MONEDA VARCHAR(3) PRIMARY KEY,
  TASA_CAMBIO FLOAT NOT NULL
);
```


Columna	Tipo	Descripción
MONEDA	VARCHAR(3)	Código de moneda. Es la clave primaria. Máximo 3 caracteres ("ARS", "USD")
TASA_CAMBIO	FLOAT	Cuántos pesos argentinos vale 1 unidad de esa moneda

Punto importante: La clave primaria es un String, no un número autoincremental. Esto tiene sentido porque el código de moneda ya es único por naturaleza.

Tabla TARJETAS

```
CREATE TABLE TARJETAS (
  ID BIGINT AUTO_INCREMENT PRIMARY KEY,
  NUMERO VARCHAR(16) NOT NULL UNIQUE,
  TITULAR VARCHAR(100) NOT NULL,
  LIMITE_CREDITO FLOAT NOT NULL
);
```

Columna	Tipo	Descripción
ID	BIGINT AUTO_INCREMENT	Identificador numérico generado automáticamente
NUMERO	VARCHAR(16)	Número del plástico. UNIQUE = no pueden repetirse
TITULAR	VARCHAR(100)	Nombre del dueño de la tarjeta
LIMITE_CREDITO	FLOAT	Monto máximo disponible por mes

Tabla CONSUMOS

```
CREATE TABLE CONSUMOS (
  ID BIGINT AUTO_INCREMENT PRIMARY KEY,
  ID_TARJETA BIGINT NOT NULL,
  MONTO FLOAT NOT NULL,
  DIA INT NOT NULL,
  MES INT NOT NULL,
  ANIO INT NOT NULL,
  RUBRO VARCHAR(20) NOT NULL,
  MONEDA VARCHAR(3) NOT NULL,
```

```

        CONSTRAINT fk_consumo_tarjeta
        FOREIGN KEY (ID_TARJETA) REFERENCES TARJETAS (ID)
    );

```

Columna	Tipo	Descripción
ID	BIGINT AUTO_INCREMENT	Identificador único del consumo
ID_TARJETA	BIGINT	Referencia a qué tarjeta realizó el consumo
MONTO	FLOAT	Cuánto se gastó
DIA/MES/ANIO	INT	Fecha del consumo separada en tres campos
RUBRO	VARCHAR(20)	Categoría del gasto (SUPERMERCADO, COMBUSTIBLE, etc.)
MONEDA	VARCHAR(3)	En qué moneda se realizó el gasto

La clave foránea `fk_consumo_tarjeta` garantiza que no pueda existir un consumo que reference a una tarjeta inexistente. Si se intenta insertar un consumo con un `ID_TARJETA` que no existe en `TARJETAS`, la base de datos rechaza la operación.

Tabla LIQUIDACIONES

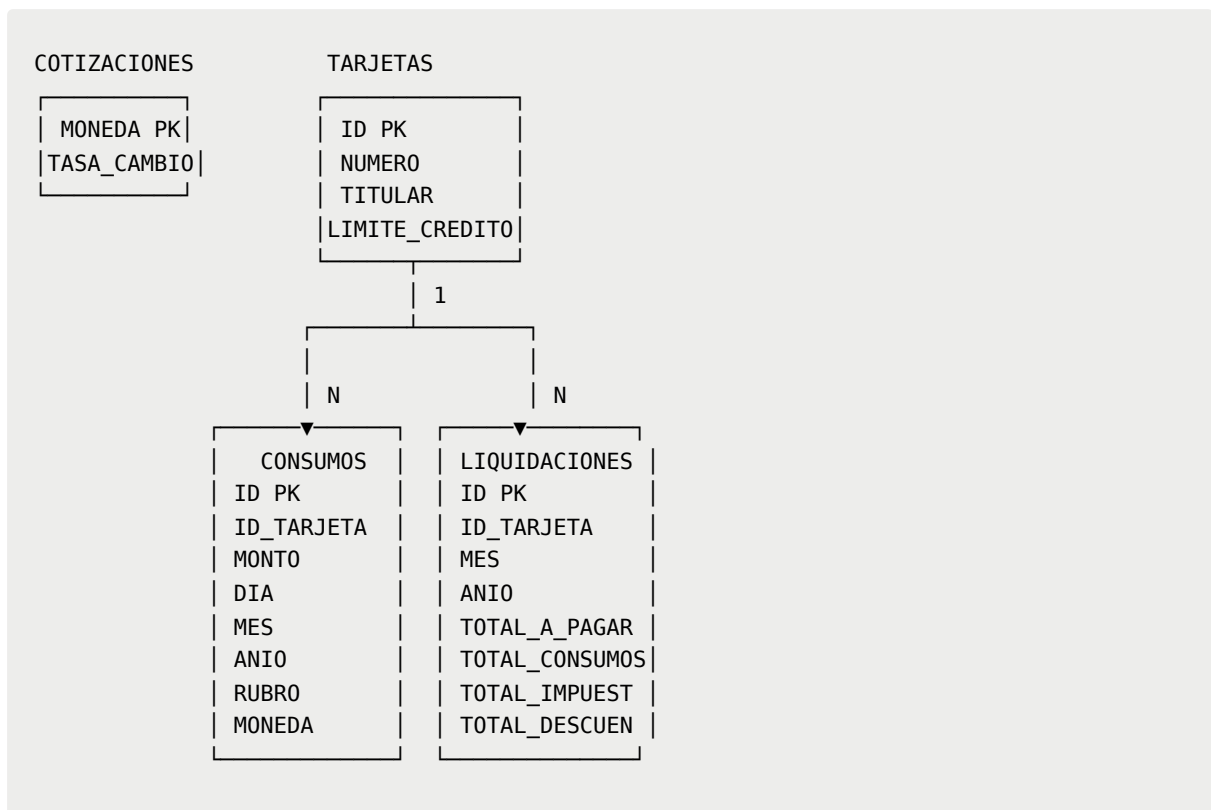
```

CREATE TABLE LIQUIDACIONES (
    ID BIGINT AUTO_INCREMENT PRIMARY KEY,
    ID_TARJETA BIGINT NOT NULL,
    MES INT NOT NULL,
    ANIO INT NOT NULL,
    TOTAL_A_PAGAR FLOAT NOT NULL,
    TOTAL_CONSUMOS FLOAT NOT NULL,
    TOTAL_IMPUESTOS FLOAT NOT NULL,
    TOTAL_DESCUENTOS FLOAT NOT NULL,
    CONSTRAINT fk_liquidacion_tarjeta
    FOREIGN KEY (ID_TARJETA) REFERENCES TARJETAS (ID)
);

```

Columna	Tipo	Descripción
ID	BIGINT AUTO_INCREMENT	Identificador único
ID_TARJETA	BIGINT	A qué tarjeta corresponde esta liquidación
MES/ANIO	INT	El período que cubre la liquidación
TOTAL_CONSUMOS	FLOAT	Suma de todos los consumos del mes en ARS
TOTAL_IMPUESTOS	FLOAT	Impuestos calculados sobre los consumos
TOTAL_DESCUENTOS	FLOAT	Descuentos aplicados
TOTAL_A_PAGAR	FLOAT	El monto final: consumos + impuestos - descuentos

Diagrama de relaciones entre tablas



5.2 `data.sql` - Datos iniciales

Este archivo se ejecuta **después** de que se crean las tablas. Carga los datos con los que trabajará la aplicación.

Cotizaciones

```
INSERT INTO COTIZACIONES (MONEDA, TASA_CAMBIO) VALUES
('ARS', 1.00),
('USD', 1550.00),
('EUR', 1680.00),
('BRL', 280.00),
('CLP', 1.45);
```

Carga 5 monedas. El ARS tiene tasa 1.00 porque es la moneda base. Un USD vale 1550 pesos, un EUR vale 1680, etc.

Tarjetas

```
INSERT INTO TARJETAS (NUMERO, TITULAR, LIMITE_CREDITO) VALUES
('4500123412340001', 'Juan Perez', 1500000.0),
('4500123412340002', 'Maria Gomez', 1200000.0),
-- ... 8 tarjetas más
```

Carga 10 tarjetas. El ID se genera automáticamente (AUTO_INCREMENT), por eso no se especifica en el INSERT.

Consumos

Los consumos se insertan en bloques separados por tarjeta:

```
-- Consumos de la tarjeta 1 (Juan Perez) - 15 consumos en Mayo 2026
INSERT INTO CONSUMOS (ID_TARJETA, MONTO, DIA, MES, ANIO, RUBRO, MONEDA) VALUES
(1, 4500.0, 1, 5, 2026, 'SUPERMERCADO', 'ARS'),
(1, 20.0, 2, 5, 2026, 'OTROS', 'USD'),
-- ...
```

Punto importante: La tarjeta 4 tiene consumos en **dos meses distintos** (Abril y Mayo 2026), lo cual es relevante para las consultas por período.

```
-- Tarjeta 4 - consumos en ABRIL
(4, 20000.0, 15, 4, 2026, 'OTROS', 'ARS'),
(4, 5000.0, 28, 4, 2026, 'COMBUSTIBLE', 'ARS'),
-- Tarjeta 4 - consumos en MAYO
(4, 35000.0, 2, 5, 2026, 'SUPERMERCADO', 'ARS'),
```

No se insertan liquidaciones en el data.sql. La tabla LIQUIDACIONES comienza vacía, lo cual es coherente con el sistema: las liquidaciones se generan por código, no se precargan.

6. Configuración JPA - `persistence.xml`

¿Qué es y dónde vive?

Es el archivo de configuración central de JPA. Debe estar ubicado **exactamente** en:

```
src/main/resources/META-INF/persistence.xml
```

JPA busca este archivo en esa ubicación específica. Si no está ahí, el sistema no arranca.

Análisis completo del archivo

Declaración de la unidad de persistencia

```
<persistence-unit name="tarjetasPU" transaction-type="RESOURCE_LOCAL">
```

Atributo	Valor	Significado
<code>name</code>	<code>tarjetasPU</code>	Nombre con el que se referencia desde el código Java
<code>transaction-type</code>	<code>RESOURCE_LOCAL</code>	Las transacciones las maneja la aplicación manualmente

El `transaction-type="RESOURCE_LOCAL"` es importante: significa que nosotros somos responsables de llamar a `begin()` y `commit()` en cada operación. La alternativa sería `JTA`, que delega el manejo de transacciones a un servidor de aplicaciones.

Registro de entidades

```
<class>ar.edu.utnfc.backend.model.Tarjeta</class>
<class>ar.edu.utnfc.backend.model.Consumo</class>
<class>ar.edu.utnfc.backend.model.Cotizacion</class>
<class>ar.edu.utnfc.backend.model.Liquidacion</class>
```

Le dice a JPA exactamente qué clases son entidades. Aunque las clases tienen la anotación `@Entity`, en un proyecto no-Spring es necesario listarlas explícitamente aquí.

Configuración de conexión

```
<property name="jakarta.persistence.jdbc.driver"
          value="org.h2.Driver"/>
<property name="jakarta.persistence.jdbc.url"
          value="jdbc:h2:mem:tarjetasdb;DB_CLOSE_DELAY=-1"/>
>
<property name="jakarta.persistence.jdbc.user"      value="sa"/>
<property name="jakarta.persistence.jdbc.password" value="" />
```

La URL `jdbc:h2:mem:tarjetasdb;DB_CLOSE_DELAY=-1` se descompone así:

```
jdbc:h2:mem:tarjetasdb;DB_CLOSE_DELAY=-1
|      |      |      |      |
|      |      |      |      └─ Mantener BD viva aunque no haya conexiones
|      |      |      └─ Nombre de la base de datos
|      |      └─ "mem" = en memoria RAM
|      └─ Motor de base de datos: H2
└─ Protocolo de conexión Java
```

Configuración de recreación de la BD

```
<property name="jakarta.persistence.schema-generation.database.action"
          value="drop-and-create"/>
```

Esta es la propiedad más crítica. Le dice a JPA que al iniciar:

1. **DROP**: Eliminar todas las tablas existentes
2. **CREATE**: Crear todas las tablas desde cero

```
<property name="jakarta.persistence.schema-generation.create-source"
          value="script"/>
<property name="jakarta.persistence.schema-generation.create-script-source"
          value="schema.sql"/>
```

Le dice que use el archivo `schema.sql` para crear las tablas (en lugar de generarlas automáticamente desde las anotaciones).

```
<property name="jakarta.persistence.sql-load-script-source"
          value="data.sql"/>
```

Después de crear las tablas, ejecuta `data.sql` para cargar los datos iniciales.

Configuración de Hibernate

```
<property name="hibernate.show_sql" value="true"/>
<property name="hibernate.format_sql" value="true"/>
<property name="hibernate.dialect" value="org.hibernate.dialect.H2Dialect"/>
```

Propiedad	Efecto
<code>show_sql</code>	Muestra en consola cada SQL que ejecuta Hibernate
<code>format_sql</code>	Lo muestra formateado y legible
<code>dialect</code>	Le dice a Hibernate qué "dialecto" SQL usar según la BD

El **dialect** es importante porque cada base de datos tiene pequeñas diferencias en su SQL. `H2Dialect` le dice a Hibernate que genere SQL compatible con H2.

7. Capa de Modelo

Las clases de modelo son la **representación en Java** de las tablas de la base de datos. Cada instancia de estas clases corresponde a una fila en la tabla correspondiente.

Anotaciones JPA fundamentales

Antes de analizar cada clase, es importante entender las anotaciones que se usan:

Anotación	Significado
<code>@Entity</code>	Esta clase representa una tabla en la BD
<code>@Table(name="X")</code>	El nombre de la tabla en la BD
<code>@Id</code>	Este campo es la clave primaria
<code>@GeneratedValue</code>	El valor se genera automáticamente
<code>@Column(name="X")</code>	El nombre de la columna en la BD
<code>@ManyToOne</code>	Relación muchos-a-uno con otra entidad
<code>@OneToMany</code>	Relación uno-a-muchos con otra entidad
<code>@JoinColumn</code>	La columna que actúa como clave foránea

Anotaciones Lombok fundamentales

Anotación	Genera
<code>@Getter</code>	Todos los métodos <code>getX()</code>
<code>@Setter</code>	Todos los métodos <code>setX()</code>
<code>@NoArgsConstructor</code>	Constructor sin parámetros
<code>@AllArgsConstructor</code>	Constructor con todos los parámetros
<code>@ToString</code>	Método <code>toString()</code>

7.1 `Cotizacion.java`

```
@Entity
@Table(name = "COTIZACIONES")
@Getter @Setter
```



```

@NoArgsConstructor
@AllArgsConstructor
@ToString
public class Cotizacion {

    @Id
    @Column(name = "MONEDA", length = 3)
    private String moneda;

    @Column(name = "TASA_CAMBIO", nullable = false)
    private Double tasaCambio;
}

```

Puntos clave

La clave primaria es un String:

```

@Id
@Column(name = "MONEDA", length = 3)
private String moneda;

```

A diferencia de las otras entidades, aquí no hay `@GeneratedValue`. El código de moneda ("ARS", "USD") ya es único y no necesita un número generado.

No tiene relaciones con otras entidades:

La tabla COTIZACIONES es independiente. Los consumos tienen una columna MONEDA pero no es una clave foránea que apunte a COTIZACIONES, es solo un texto. Esto simplifica la entidad.

7.2 Tarjeta.java

```

@Entity
@Table(name = "TARJETAS")
@Getter @Setter
@NoArgsConstructor
@AllArgsConstructor
@ToString(exclude = {"consumos", "liquidaciones"})
public class Tarjeta {

```

```

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = "ID")
private Long id;

@Column(name = "NUMERO", length = 16, nullable = false,
unique = true)
private String numero;

@Column(name = "TITULAR", length = 100, nullable = false)
private String titular;

@Column(name = "LIMITE_CREDITO", nullable = false)
private Double limiteCredito;

@OneToMany(mappedBy = "tarjeta", cascade = CascadeType.
ALL, orphanRemoval = true)
private List<Consumo> consumos = new ArrayList<>();

@OneToMany(mappedBy = "tarjeta", cascade = CascadeType.
ALL, orphanRemoval = true)
private List<Liquidacion> liquidaciones = new ArrayList
<>();
}

```

Puntos clave

Generación de ID:

```

@GeneratedValue(strategy = GenerationType.IDENTITY)

```

IDENTITY le dice a Hibernate que use el AUTO_INCREMENT de la base de datos para generar el ID. Cuando se hace `persist()`, la BD asigna el siguiente número disponible.

La anotación `unique = true`:

```
@Column(name = "NUMERO", length = 16, nullable = false, unique = true)
```

Esto hace que Hibernate genere una restricción UNIQUE en la base de datos, garantizando que no puedan existir dos tarjetas con el mismo número.

Las relaciones OneToMany:

```
@OneToMany(mappedBy = "tarjeta", cascade = CascadeType.ALL, orphanRemoval = true)
private List<Consumo> consumos = new ArrayList<>();
```

- `mappedBy = "tarjeta"`: Le dice a JPA que la relación ya está definida en el campo `tarjeta` de la clase `Consumo`. Tarjeta no es la dueña de la relación.
- `cascade = CascadeType.ALL`: Si se elimina una tarjeta, se eliminan automáticamente todos sus consumos.
- `orphanRemoval = true`: Si se quita un consumo de la lista, se elimina de la BD.

El `@ToString(exclude = ...)`:

```
@ToString(exclude = {"consumos", "liquidaciones"})
```

Se excluyen las listas para evitar un **bucle infinito**: si `toString()` de Tarjeta llama a `toString()` de Consumo, y `toString()` de Consumo llama a `toString()` de Tarjeta, el programa entraría en un loop sin fin.

7.3 Consumo.java

```
@Entity
@Table(name = "CONSUMOS")
@Getter @Setter
@NoArgsConstructor
@AllArgsConstructor
@ToString(exclude = "tarjeta")
public class Consumo {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```

@Column(name = "ID")
private Long id;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "ID_TARJETA", nullable = false)
private Tarjeta tarjeta;

@Column(name = "MONT0", nullable = false)
private Double monto;

@Column(name = "DIA", nullable = false)
private Integer dia;

@Column(name = "MES", nullable = false)
private Integer mes;

@Column(name = "ANIO", nullable = false)
private Integer anio;

@Column(name = "RUBRO", length = 20, nullable = false)
private String rubro;

@Column(name = "MONEDA", length = 3, nullable = false)
private String moneda;
}

```

Puntos clave

La relación ManyToOne:

```

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "ID_TARJETA", nullable = false)
private Tarjeta tarjeta;

```

Muchos consumos pertenecen a una tarjeta. `@JoinColumn(name = "ID_TARJETA")` indica que la columna `ID_TARJETA` en la tabla CONSUMOS es la clave foránea que apunta a TARJETAS.

FetchType.LAZY:

```
@ManyToOne(fetch = FetchType.LAZY)
```

Significa **carga diferida**. Cuando se carga un Consumo, Hibernate NO carga automáticamente la Tarjeta asociada. Solo la carga cuando el código realmente la necesita (cuando se llama a `consumo.getTarjeta()`). Esto mejora el rendimiento porque evita cargar datos innecesarios.

La alternativa sería `FetchType.EAGER`, que cargaría la Tarjeta siempre, aunque no se necesite.

La fecha separada en tres campos:

```
private Integer dia;  
private Integer mes;  
private Integer anio;
```

En lugar de usar un tipo `Date` o `LocalDate`, el diseño de la BD usa tres enteros separados. Esto facilita las consultas por mes y año sin necesidad de funciones de fecha.

7.4 Liquidacion.java

```
@Entity  
@Table(name = "LIQUIDACIONES")  
@Getter @Setter  
@NoArgsConstructor  
@AllArgsConstructor  
@ToString(exclude = "tarjeta")  
public class Liquidacion {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    @Column(name = "ID")  
    private Long id;  
  
    @ManyToOne(fetch = FetchType.LAZY)  
    @JoinColumn(name = "ID_TARJETA", nullable = false)  
    private Tarjeta tarjeta;
```

```

@Column(name = "MES", nullable = false)
private Integer mes;

@Column(name = "ANIO", nullable = false)
private Integer anio;

@Column(name = "TOTAL_A_PAGAR", nullable = false)
private Double totalAPagar;

@Column(name = "TOTAL_CONSUMOS", nullable = false)
private Double totalConsumos;

@Column(name = "TOTAL_IMPUESTOS", nullable = false)
private Double totalImpuestos;

@Column(name = "TOTAL_DESCUENTOS", nullable = false)
private Double totalDescuentos;
}

```

Puntos clave

Los cuatro montos:

Una liquidación desglosa el total en partes:

```
totalAPagar = totalConsumos + totalImpuestos - totalDescuentos
```

Sin datos en data.sql:

La tabla LIQUIDACIONES no tiene datos precargados. Las liquidaciones se crean mediante código cuando se procesa el mes de una tarjeta.

8. Capa de Repositorio

Los repositorios son las clases que **concentran todo el acceso a la base de datos**. Ninguna otra clase debería interactuar directamente con el `EntityManager`. Esta separación es un patrón de diseño llamado **Repository Pattern**.

El EntityManager

Antes de analizar los repositorios, es fundamental entender qué es el

`EntityManager`:

`EntityManager` = la puerta de entrada a JPA

Es el objeto que:

- Guarda entidades en la BD (`persist`)
- Busca entidades por ID (`find`)
- Ejecuta consultas JPQL (`createQuery`)
- Maneja transacciones (`getTransaction`)
- Sincroniza el estado de los objetos Java con la BD (`merge`)

Todos los repositorios reciben el `EntityManager` por constructor:

```
public TarjetaRepository(EntityManager em) {  
    this.em = em;  
}
```

Esto se llama **inyección por constructor** y facilita los tests porque se puede pasar cualquier `EntityManager`.

Las transacciones

Cada operación que **modifica** datos (guardar, actualizar, eliminar) debe estar dentro de una transacción:

```
em.getTransaction().begin();    // Iniciar transacción  
em.persist(tarjeta);           // Operación  
em.getTransaction().commit();  // Confirmar cambios
```

Si algo falla entre `begin()` y `commit()`, los cambios no se aplican. Las operaciones de **solo lectura** (buscar) no necesitan transacción.

JPQL vs SQL

Los repositorios usan **JPQL** (Jakarta Persistence Query Language), no SQL directo. La diferencia es:

```
-- SQL (habla de tablas y columnas)
SELECT * FROM TARJETAS WHERE ID = 1

-- JPQL (habla de clases y atributos Java)
SELECT t FROM Tarjeta t WHERE t.id = 1
```

JPQL es independiente de la base de datos. Si mañana cambiamos de H2 a PostgreSQL, las consultas JPQL siguen funcionando sin cambios.

8.1 TarjetaRepository.java

CRUD básico

guardar:

```
public void guardar(Tarjeta tarjeta) {
    em.getTransaction().begin();
    em.persist(tarjeta);
    em.getTransaction().commit();
}
```

`persist()` le dice a JPA "este objeto nuevo debe guardarse en la BD". Después del `commit()`, el objeto tendrá su ID asignado por la BD.

actualizar:

```
public Tarjeta actualizar(Tarjeta tarjeta) {
    em.getTransaction().begin();
    Tarjeta resultado = em.merge(tarjeta);
    em.getTransaction().commit();
    return resultado;
}
```

`merge()` se usa cuando el objeto ya existe en la BD y queremos actualizar sus datos. Devuelve una nueva instancia del objeto actualizado.

eliminar:

```
public void eliminar(Long id) {
    em.getTransaction().begin();
```



```

Tarjeta tarjeta = em.find(Tarjeta.class, id);
if (tarjeta != null) {
    em.remove(tarjeta);
}
em.getTransaction().commit();
}

```

Primero se busca la entidad con `find()` para obtener una instancia **manejada** por JPA, y luego se elimina. No se puede llamar `remove()` directamente con un objeto que no esté siendo rastreado por JPA.

buscarTodas:

```

public List<Tarjeta> buscarTodas() {
    return em.createQuery("SELECT t FROM Tarjeta t", Tarjeta.class)
        .getResultList();
}

```

Consulta JPQL simple. `Tarjeta.class` como segundo parámetro le dice a JPA el tipo de resultado esperado, lo que permite devolver una lista tipada.

Consultas específicas

buscarSinLiquidacion:

```

public List<Tarjeta> buscarSinLiquidacion(int anio, int meses) {
    String jpql = ""
        + "SELECT t FROM Tarjeta t"
        + "WHERE t.id NOT IN ("
        + "    SELECT l.tarjeta.id FROM Liquidacion l"
        + "    WHERE l.anio = :anio AND l.mes = :mes"
        + ")";
    return em.createQuery(jpql, Tarjeta.class)
        .setParameter("anio", anio)
        .setParameter("mes", meses);
}

```

```
        .getResultList();
    }
}
```

Esta consulta usa una **subconsulta**:

1. La subconsulta obtiene los IDs de tarjetas que SÍ tienen liquidación en ese período
2. La consulta principal devuelve las tarjetas cuyos IDs NO están en esa lista

Los `:anio` y `:mes` son **parámetros nombrados**. Se asignan con `setParameter()`. Esto evita la **inyección SQL**, que es una vulnerabilidad de seguridad.

buscarPorNumero:

```
public Optional<Tarjeta> buscarPorNumero(String numero) {
    try {
        Tarjeta t = em.createQuery(
            "SELECT t FROM Tarjeta t WHERE t.numero = :
numero",
            Tarjeta.class)
            .setParameter("numero", numero)
            .getSingleResult();
        return Optional.of(t);
    } catch (NoResultException e) {
        return Optional.empty();
    }
}
```

`getSingleResult()` lanza una excepción `NoResultException` si no encuentra nada, en lugar de devolver null. Por eso se usa un try-catch y se devuelve `Optional.empty()` en ese caso.

`Optional<T>` es una clase de Java que representa "puede haber un valor o puede no haberlo", evitando los `NullPointerException`.

8.2 ConsumoRepository.java

Consulta específica principal

buscarPorTarjetaAnioMes:

```

public List<Consumo> buscarPorTarjetaAnioMes(Long idTarjeta,
int anio, int mes) {
    String jpql = ""
        SELECT c FROM Consumo c
        WHERE c.tarjeta.id = :idTarjeta
            AND c.anio = :anio
            AND c.mes = :mes
        ORDER BY c.dia
        "";
    return em.createQuery(jpql, Consumo.class)
        .setParameter("idTarjeta", idTarjeta)
        .setParameter("anio", anio)
        .setParameter("mes", mes)
        .getResultList();
}

```

Puntos importantes:

- `c.tarjeta.id`: JPQL permite navegar por las relaciones con punto. Esto se traduce automáticamente a un JOIN con la tabla TARJETAS.
- `ORDER BY c.dia`: Los resultados vienen ordenados por día del mes.

8.3 CotizacionRepository.java

Similar a los otros repositorios pero con una diferencia importante: la clave primaria es un `String` en lugar de un `Long`.

```

public Optional<Cotizacion> buscarPorMoneda(String moneda)
{
    return Optional.ofNullable(em.find(Cotizacion.class, moneda));
}

public void eliminar(String moneda) {
    em.getTransaction().begin();
    Cotizacion cotizacion = em.find(Cotizacion.class, moneda);
}

```

```
// ...  
}
```

`em.find(Cotizacion.class, moneda)` funciona igual que con un Long, JPA simplemente usa ese String como clave primaria.

8.4 LiquidacionRepository.java

Consulta específica principal

buscarPorNumeroTarjetaAnioMes:

```
public Optional<Liquidacion> buscarPorNumeroTarjetaAnioMes(  
    String numeroTarjeta, int anio, int mes) {  
    try {  
        Liquidacion l = em.createQuery("""  
            SELECT l FROM Liquidacion l  
            WHERE l.tarjeta.numero = :numero  
            AND l.anio = :anio  
            AND l.mes = :mes  
            """, Liquidacion.class)  
            .setParameter("numero", numeroTarjeta)  
            .setParameter("anio", anio)  
            .setParameter("mes", mes)  
            .getSingleResult();  
        return Optional.of(l);  
    } catch (NoResultException e) {  
        return Optional.empty();  
    }  
}
```

Esta consulta es interesante porque busca por `l.tarjeta.numero`, es decir, navega desde Liquidacion hasta Tarjeta para comparar el número. Hibernate traduce esto a un JOIN automático:

```
-- Lo que Hibernate genera internamente  
SELECT l.* FROM LIQUIDACIONES l  
JOIN TARJETAS t ON l.ID_TARJETA = t.ID  
WHERE t.NUMERO = ?
```

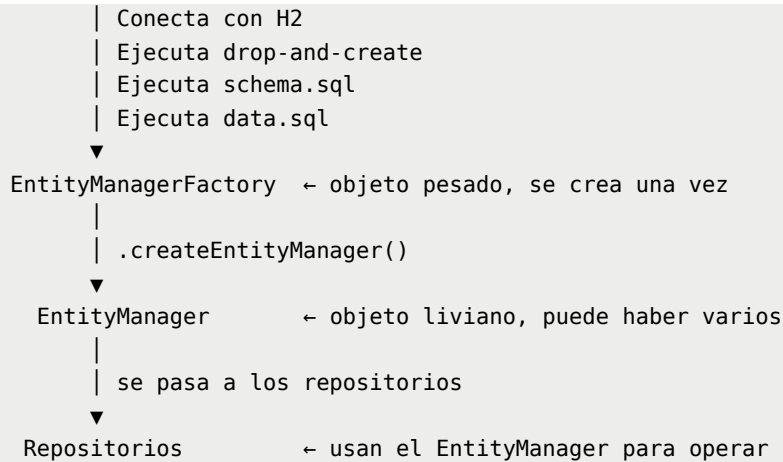
```
AND l.ANIO = ?  
AND l.MES = ?
```

9. Punto de entrada - **Main.java**

```
public class Main {  
    public static void main(String[] args) {  
  
        // 1. Crear la fábrica de EntityManagers  
        EntityManagerFactory emf =  
            Persistence.createEntityManagerFactory("tarjeta  
sPU");  
  
        // 2. Crear el EntityManager  
        EntityManager em = emf.createEntityManager();  
  
        // 3. Crear repositorios  
        TarjetaRepository tarjetaRepo = new TarjetaReposito  
ry(em);  
        // ...  
  
        // 4. Usar repositorios  
        tarjetaRepo.buscarTodas().forEach(System.out::print  
ln);  
  
        // 5. Cerrar recursos  
        em.close();  
        emf.close();  
    }  
}
```

El ciclo de vida de JPA

```
Persistence.createEntityManagerFactory("tarjetasPU")  
    |  
    | Lee persistence.xml
```



Por qué se cierran al final:

```
em.close();    // Libera la conexión a la BD
emf.close();   // Cierra la fábrica y libera todos los recursos
```

No cerrarlos causaría **memory leaks** (pérdidas de memoria) porque los recursos de conexión quedarían ocupados indefinidamente.

10. Tests - TarjetasTest.java

Estructura general

```
@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
class TarjetasTest {

    private static EntityManagerFactory emf;
    private static EntityManager em;

    @BeforeAll
    static void setUp() {
        // Se ejecuta UNA VEZ antes de todos los tests
        emf = Persistence.createEntityManagerFactory("tarjetasPU");
        em = emf.createEntityManager();
        // Aquí también se recrea la BD
    }
}
```

```

    }

    @AfterAll
    static void tearDown() {
        // Se ejecuta UNA VEZ después de todos los tests
        em.close();
        emf.close();
    }
}

```

Ciclo de vida de los tests

```

@BeforeAll setUp()
| Crea EMF y EM
| Recrea la BD con schema.sql y data.sql
▼
@Test @Order(1) testBuscarTodasLasCotizaciones
▼
@Test @Order(2) testBuscarCotizacionPorMoneda
▼
... (todos los tests en orden)
▼
@Test @Order(12) testBuscarLiquidacionInexistente
▼
@AfterAll tearDown()
| Cierra EM y EMF

```

Por qué el orden importa

Los tests están ordenados con `@Order` porque algunos **dependen del estado creado por tests anteriores**:

```

Test 10: Verifica que hay 10 tarjetas sin liquidación
↓ (no modifica nada)
Test 11: Crea una liquidación para la tarjeta 1
↓ (ahora hay 1 liquidación en la BD)
Test 11: Verifica que ahora hay 9 tarjetas sin liquidación
↓
Test 12: Verifica que la tarjeta 2 NO tiene liquidación

```

Si el test 11 se ejecutara antes que el 10, el test 10 fallaría porque ya habría una liquidación en la BD.

Análisis de tests importantes

Test de subconsulta:

```
@Test @Order(10)
void testTarjetasSinLiquidacionMayo2026() {
    List<Tarjeta> sinLiq = tarjetaRepo.buscarSinLiquidacion
(2026, 5);
    assertEquals(10, sinLiq.size());
    // Sin liquidaciones cargadas, las 10 tarjetas deben ap
    arecer
}
```

Test de estado encadenado:

```
@Test @Order(11)
void testGuardarLiquidacionYBuscarPorNumero() {
    Tarjeta t1 = tarjetaRepo.buscarPorId(1L).orElseThrow();

    // Crear y guardar liquidación
    Liquidacion liq = new Liquidacion(null, t1, 5, 2026,
                                     150000.0, 130000.0, 2
0000.0, 0.0);
    liquidacionRepo.guardar(liq);

    // Verificar que el ID fue asignado
    assertNotNull(liq.getId());

    // Verificar que ahora son 9 sin liquidación
    assertEquals(9, tarjetaRepo.buscarSinLiquidacion(2026,
5).size());

    // Verificar búsqueda por número
    Optional<Liquidacion> encontrada =
        liquidacionRepo.buscarPorNumeroTarjetaAnioMes(
            "4500123412340001", 2026, 5);
    assertTrue(encontrada.isPresent());
    assertEquals(150000.0, encontrada.get().getTotalAPagar
```



```
());  
}
```

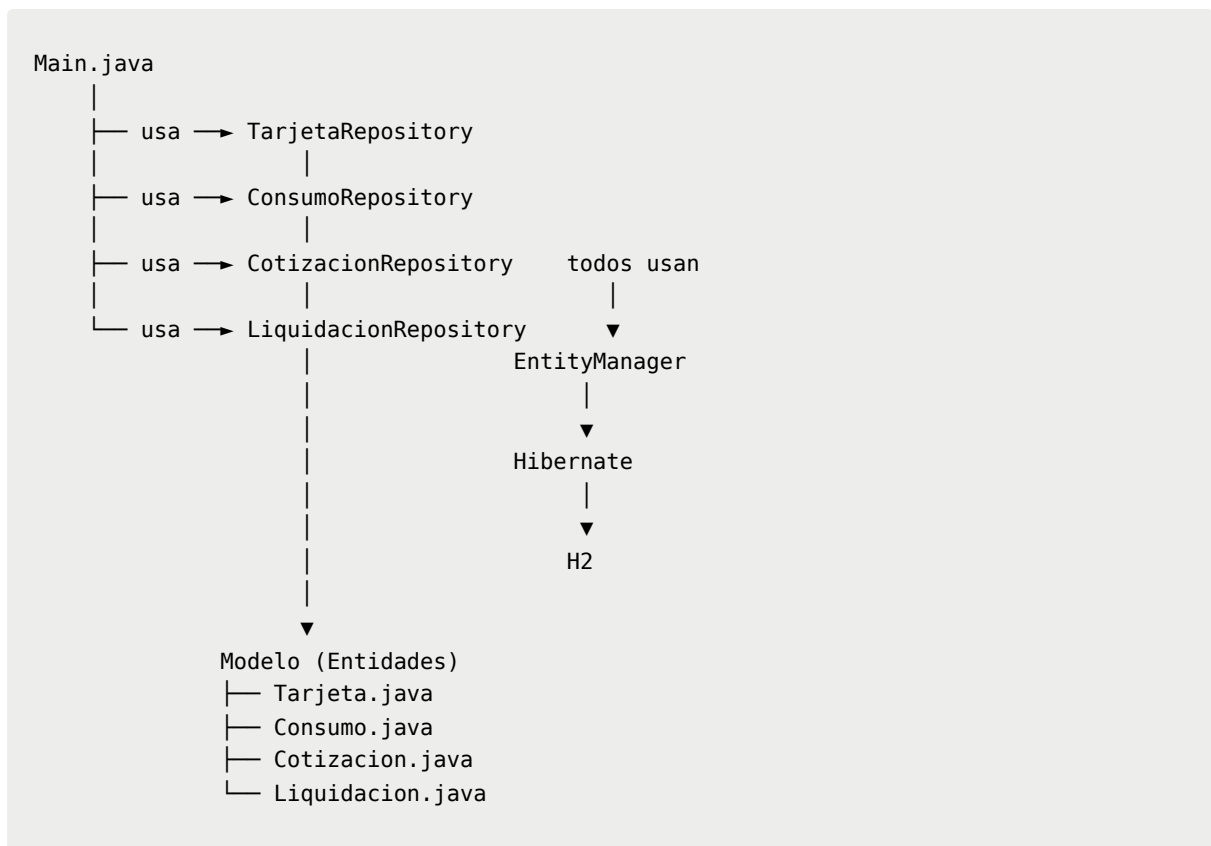
11. Flujo completo de ejecución

Desde que se ejecuta el programa hasta que termina

```
1. JVM inicia Main.java  
   |  
2. Persistence.createEntityManagerFactory("tarjetasPU")  
   |  
   |— Lee META-INF/persistence.xml  
   |— Carga el driver H2  
   |— Conecta a jdbc:h2:mem:tarjetasdb  
   |— Ejecuta DROP de todas las tablas (si existen)  
   |— Ejecuta schema.sql → crea COTIZACIONES, TARJETAS, CONSUMOS, LIQUIDACIONES  
   |— Ejecuta data.sql → inserta 5 cotizaciones, 10 tarjetas, ~80 consumos  
   |  
3. emf.createEntityManager()  
   |  
   |— Crea el objeto que gestiona las operaciones JPA  
   |  
4. new TarjetaRepository(em) (y demás repositorios)  
   |  
   |— Los repositorios reciben el EntityManager  
   |  
5. tarjetaRepo.buscarTodas()  
   |  
   |— Hibernate genera: SELECT t1_0.ID, t1_0.LIMITE_CREDITO,  
   |                       t1_0.NUMERO, t1_0.TITULAR  
   |                       FROM TARJETAS t1_0  
   |  
   |— H2 ejecuta el SQL  
   |— H2 devuelve 10 filas  
   |— Hibernate convierte cada fila en un objeto Tarjeta  
   |  
6. System.out.println() muestra los resultados  
   |  
7. em.close() → libera la conexión  
8. emf.close() → la BD en memoria desaparece
```

12. Relaciones entre componentes

Diagrama de dependencias



Resumen de responsabilidades

Archivo	Responsabilidad
<code>pom.xml</code>	Define dependencias y configuración de compilación
<code>persistence.xml</code>	Configura JPA, conexión a BD y recreación de esquema
<code>schema.sql</code>	Define la estructura de las tablas
<code>data.sql</code>	Carga los datos iniciales
<code>Cotizacion.java</code>	Representa una tasa de cambio
<code>Tarjeta.java</code>	Representa una tarjeta de crédito con sus relaciones
<code>Consumo.java</code>	Representa un gasto realizado con una tarjeta
<code>Liquidacion.java</code>	Representa el resumen mensual de pagos
<code>TarjetaRepository.java</code>	CRUD de tarjetas + consultas especiales
<code>ConsumoRepository.java</code>	CRUD de consumos + búsqueda por período
<code>CotizacionRepository.java</code>	CRUD de cotizaciones
<code>LiquidacionRepository.java</code>	CRUD de liquidaciones + búsqueda por número y período

Archivo	Responsabilidad
Main.java	Punto de entrada, inicializa JPA y demuestra funcionalidad
TarjetasTest.java	Verifica que todo el sistema funciona correctamente